



CERTIFICATE PROGRAM IN ALGORITHMIC TRADING (CPAT)

Capstone Project

Project Title: Development and Implementation of a Complete Multi-Indicator Multi-Asset Algorithmic Live Trading System for Equity Markets

Assignment Date: 25th April, 2026

Last Date of submission: 30th May, 2026

A. Project Objective

This project aims to help students learn to design, develop, rigorously back-test, conduct performance analysis, optimize, risk-manage, and deploy in the live-market integrated with a broker's API a fully automated algorithmic trading strategy that combines multiple technical indicators for signal generation. The system must demonstrate end-to-end proficiency in the algorithmic trading lifecycle.

B. Project Scope & Requirements

Students must deliver a complete, self-contained Python-based algorithmic trading system covering:

1. Strategy Development (Modules 301, 302, 303)

- Propose **one primary strategy** and **one variant** (e.g., a momentum or mean-reversion strategy).
- **Recommended base ideas** (student can choose or propose their own):
 - Dual Moving Average Crossover with Volume confirmation (trend-following + momentum)
 - Bollinger Bands + RSI mean-reversion strategy
 - Breakout strategy based on volatility (ATR)
- Incorporate at least **2–3 technical indicators** using **TA-Lib**.

2. Data Handling (Module 304)

- Fetch historical data for at least **50 stocks and one index (e.g. Nifty 50)** using a broker's API (e.g. Alpaca or any Indian broker) or Yahoo finance.
- Build a local data store implement data cleaning and regular updating system.

3. Back-testing Engine, Performance Analysis and Optimization (Module 305, 306, 307)

- Build a custom **back-testing** framework from scratch.
- Implement realistic elements:
 - Transaction costs (0.1%–0.3% round-trip brokerage + slippage)



- Avoid common pitfalls: look-ahead bias, survivorship bias, overfitting
- Calculate comprehensive **performance metrics**: Cumulative returns, CAGR, Sharpe Ratio, Sortino Ratio, Max Drawdown, Win Rate, Profit Factor, Calmar Ratio, etc.
- Perform **optimization**.

4. Money Management & Risk Management (Module 308)

- Implement position sizing (e.g., Kelly criterion, or volatility-based).
- Apply risk controls: Stop-loss, maximum daily loss limit, portfolio-level maximum loss limit.
- Optimal capital allocation across multiple assets.

5. Algorithmic Trading Infrastructure & System Design (Modules 309 & 310)

- Design the system architecture (modular code structure: data module, signal generation, order management, risk module, logging).
- Implement **order management system** (simulated market orders, limit orders).
- Add **error handling**, logging, and robustness features (e.g., handling API failures, data gaps).
- Integrate with a broker API for **live/paper trading** modes.

6. Visualization & Reporting (Optional)

- Generate reports and plots for back test and live/paper trading performance:
 - Equity curve
 - Drawdown chart
 - Indicator + signal visualization on price chart

C. Deliverables (What Students Must Submit)

- **Complete Python codebase** (well-structured, modular, commented.)
- **Detailed Project Report**:
 - Strategy logic and rationale
 - Back-testing methodology and bias checks
 - Optimization process and results
 - Risk management rules
 - Performance analysis with metrics and charts
 - Limitations, lessons learned, and potential improvements
- **Live demonstration (optional but encouraged)**: Run the strategy in simulation/paper-trading mode during review



D. Evaluation Criteria (to assess learning outcomes)

- **Technical Implementation (40%):** Clean, efficient, from-scratch Python code covering strategy coding, back-testing, optimization and live-trading system design.
- **Strategy & Back-testing Rigor (25%):** Avoidance of biases, realistic assumptions, proper optimization, and performance measurement.
- **Risk & Money Management (15%):** Sound risk controls and capital allocation.
- **Infrastructure & Robustness (10%):** Modular design, error handling, and deployment awareness.
- **Report & Presentation (10%):** Clarity, depth of analysis, and professional communication.

This capstone project evaluates whether students can:

- Code their own strategies in Python
- Build and rigorously test algorithms while avoiding pitfalls
- Apply risk and money management
- Design and implement a deployable trading system

Minimum Passing Requirement: Working code that produces valid back-test results with documented risk controls + proper working paper-trading system + submission of report.